Fleshfetch
Modding Reference
Complete documentation for the current mod API

1. Mod Structure

Every mod is a subfolder inside the mods/ directory. Raw .py files in the mods/ folder are ignored. The folder name is the mod internal ID. Required and optional files inside a mod folder:

- mod.py - required. Must define a register(game) function.
- manifest.json - optional. Metadata, dependencies, and optional title metadata.
- assets/ - optional. Folder for images, sounds, and other mod files.
- enabled.txt - optional. Created automatically if missing. Use true to load the mod and false to keep it installed but disabled.

manifest.json

The manifest supplies display metadata for the Settings mod list. It can also declare dependencies and offer a custom game title. It does not enable or disable mods. Use enabled.txt for that.

```
{
  "name": "My Mod",
  "version": "1.0.0",
  "description": "What this mod does.",
  "author": "yourname",
  "dependencies": [
    {"id": "other_mod", "version": "1.0.0", "optional": false}
  ],
  "game_title": "My Custom Title"
}
```

enabled.txt

The Settings tab lists installed mods and writes enabled.txt when the player clicks Enable or Disable. The expandable list shows name, author, version, description, dependencies, security status, shared APIs, deprecation warnings, and any title option from manifest.json. The manifest enabled field is ignored.

```
true
```

mod.py

The entry point for your mod. The game calls register(game) during mod loading. MOD_DIR and MANIFEST are injected as module-level variables before register() is called.

```
def register(game):
    # all mod setup goes here
    pass
```

Mod Load Order

Mods are loaded alphabetically by folder name. System mods are loaded before user mods. User mods are stored in %APPDATA% \fleshfetch\mods on Windows and ~/.config/fleshfetch/mods on Linux. If enabled.txt contains false, the mod is skipped but still shown in Settings. Missing dependencies and security warnings can pause a mod before register() runs.

Current App Layout

The game is modularized. The packaged app starts from main.py, and fleshfetch.py is only a legacy launcher. Mods still receive the same game object in register(game), so folder-based mods do not need to import the application modules directly.

```
fleshfetch/
  main.py
  ui.py
  save_manager.py
  mod_api.py
  security.py
  rpc.py
  paths.py
  defaults.py
  leaderboard.py
  bootstrap.py
  console_capture.py
```

2. Security Warnings

The mod security scanner is a warning system, not a sandbox. It helps players notice suspicious code before it runs, but it cannot prove that a mod is safe. Only run mods from sources you trust.

Level	Meaning
0: Safe	No warning reasons were found. The mod loads normally if enabled.
1: Suspicious	The mod uses features that can be normal or harmful, such as requests, socket, subprocess, dynamic imports, dynamic attribute access, or broad file access. The mod name is highlighted yellow.
2: Extremely Suspicious	The mod uses very risky patterns, such as keyboard capture, base64 plus execution, remote code execution, or ctypes with process APIs. The mod name is highlighted red.

Mods with Level 1 or Level 2 warnings are paused before loading until the player dismisses the warning. The dismiss popup says:

```
Are you sure you want to dismiss this warning? You might be running
very harmful code, or just a normal mod. Only continue if you trust
the mod and its source.
```

The popup buttons are Cancel and I don't care. If Auto reload mod changes is enabled, dismissing the warning reloads the mods immediately. Otherwise, run /reload or restart Fleshfetch to load the mod.

3. Dependencies

Mods can depend on other mods. Dependencies can be declared in manifest.json or checked from code. Missing required dependencies pause the mod before loading and show the missing dependency in Settings.

manifest dependency fields

The loader accepts dependencies, depends, requires_mods, or required_mods. Each dependency can be a string ID or an object with id, version, and optional.

```
{
  "dependencies": [
    "core_library",
    {"id": "plants", "version": "1.2.0", "optional": false},
    {"id": "cosmetics", "optional": true}
  ]
}
```

has_mod(mod_id, min_version="")

Returns True if a loaded mod with that ID is present. If min_version is supplied, the loaded mod version must meet or exceed it using a simple numeric version comparison.

```
if game.has_mod("plants", "1.2.0"):
    print("Plant features available")
```

require_mod(mod_id, min_version="")

Requires another loaded mod. If it is missing, the current mod is paused and listed as dependency blocked in Settings.

```
def register(game):
    game.require_mod("plants", "1.2.0")
    # setup that needs plants starts here
```

get_loaded_mods(), get_installed_mods(), get_mod_info(mod_id)

These helpers let a mod inspect loaded and installed mods. get_loaded_mods() only returns mods that actually loaded. get_installed_mods() also includes disabled, security-blocked, and dependency-blocked mods.

4. Shared Mod APIs

A mod can register a shared API object for other mods to use. Shared APIs have a normalized name, optional version, description, aliases, and an owning mod. The Settings mod list shows APIs provided by each loaded mod.

register_api(name, api, version="", description="", aliases=None, replace=False)

Registers an object under a shared API name and returns the object. replace=True allows an existing registration to be replaced. Aliases are normalized the same way as the primary API name.

```
class ParticleAPI:
    def spawn(self, x, y):
        print("particle at", x, y)

def register(game):
    game.register_api(
        "particles",
        ParticleAPI(),
        version="1.0.0",
```

```
    description="Simple particle helpers",
    aliases=["fx"],
)
```

get_api(name, default=None), get_api_info(name)
get_api() returns the registered object or default. get_api_info() returns metadata without exposing the object itself.

has_api(name, min_version=""), require_api(name, min_version="")
has_api() checks whether an API exists and optionally meets a minimum version. require_api() returns the object or raises a dependency-style error.

```
particles = game.require_api("particles", "1.0.0")
particles.spawn(20, 40)
```

list_apis(), unregister_api(name)
list_apis() returns metadata for every registered shared API. unregister_api() removes an API and its aliases. If the API object has cleanup(), shutdown(), or destroy(), the first available cleanup method is called automatically.

5. Currencies
Currencies are the core resources of the game. Flesh is the built-in currency. Mods can register additional currencies. Each currency has a registry name used in code and a display name shown to the player.

A currency amount label and per-second label only appear in the UI once the player has obtained at least 1 of it. Flesh is always shown unless a mod replaces vanilla content.

register_currency(registry_name, display_name)
Registers a new currency. Safe to call even if the currency already exists. If the currency is missing from the active save, it is added with an amount of 0.
- registry_name - string. Used in all code references. Must be unique. Example: "bones"
- display_name - string. Shown in the stats panel and upgrade costs. Example: "Bones"

```
def register(game):
    game.register_currency("bones", "Bones")
```

get_currency(registry_name)
Returns the player current amount of a currency as a float.

```
amount = game.get_currency("bones")
```

set_currency(registry_name, value)
Sets a currency to an exact value. Clamps to 0 if value is negative and marks the save dirty.

```
game.set_currency("bones", 100.0)
```

add_currency(registry_name, amount)
Adds an amount to a currency. Use a negative amount to subtract.

```
game.add_currency("bones", 10.0)
game.add_currency("bones", -5.0)
```

6. Upgrades
Upgrades are purchasable items that modify the game. Each upgrade has a cost, a cost currency, and a list of currency effects that define what it does.

register_upgrade(uid, data)
Registers a new upgrade, or updates an existing one if the uid already exists.
- uid - string. Unique internal ID for this upgrade. Example: "bone_grinder"
- data - dict. See the upgrade schema below.

Upgrade Schema

```
{
  "name": str,
  "desc": str,
  "category": str,
  "base_cost": float,
  "cost_mult": float,
  "cost_currency": str,
```

```
  "currency_effects": [
    {
      "currency": str,
      "cpc": float,
      "cps": float,
      "on_buy": float
    }
  ]
}
```

Examples

An upgrade that costs flesh and produces bones per second:

```
game.register_upgrade("bone_grinder", {
    "name": "Bone Grinder",
    "desc": "+1 bone/sec.",
    "category": "auto",
    "base_cost": 50,
    "cost_mult": 1.2,
    "cost_currency": "flesh",
    "currency_effects": [
        {"currency": "bones", "cpc": 0.0, "cps": 1.0, "on_buy": 0.0},
    ],
})
```

An upgrade that costs bones, gives bones per click, and gives 5 bones immediately on purchase:

```
game.register_upgrade("sharp_bones", {
    "name": "Sharp Bones",
    "desc": "+2 bones/click. Gives 5 bones on purchase.",
    "category": "click",
    "base_cost": 30,
    "cost_mult": 1.25,
    "cost_currency": "bones",
    "currency_effects": [
        {"currency": "bones", "cpc": 2.0, "cps": 0.0, "on_buy": 5.0},
    ],
})
```

Legacy fps and fpc keys are still accepted, but the mod is marked as legacy and receives a deprecation warning. Use currency_effects with cpc and cps. Legacy type is also accepted when category is missing, but category is preferred.

Built-in Critical Clicks

Critical Clicks is implemented as a built-in click modifier. Each owned Critical Clicks upgrade gives +5 percent chance to double the final click gains.

7. Click Modifiers

Click modifiers let mods change the currency gained from a click before the game awards it. Use them for effects like random double rewards, flat bonus rewards, or special click rules.

register_click_multiplier(modifier_id=None, chance=1.0, multiplier=2.0, currency=None, description="")

Registers a simple chance-based multiplier. chance can be 0.1 for 10 percent or 10 for 10 percent. If currency is omitted, the modifier applies to all click gains.

```
def register(game):
    game.register_click_multiplier(
        "lucky_click",
        chance=0.1,
        multiplier=2.0,
        description="10 percent chance to double click gains",
    )
```

register_click_modifier(modifier_id=None, callback=None, description="", currency=None)

Registers a callback that receives a context dict and can return a number or a dict. Returning None or False leaves the click unchanged. Returning a number multiplies the targeted gains by that number.
The context dict includes game, gains, base_gains, x, y, n_press, total_clicks, vanilla_multiplier, crit_multiplier, and modifier.
A returned dict can use these fields: multiplier, add, bonus, amount, set, currency, gains, add_gains, bonus_gains, message, and triggered.

```
def register(game):
    def lucky_bonus(ctx):
        if ctx["total_clicks"] % 25 == 0:
            return {"add": 100, "message": "25th click bonus"}
        return None

    game.register_click_modifier(
        "every_25_clicks",
        lucky_bonus,
        description="Every 25th click adds 100 to each clicked currency.",
    )
```

add_click_multiplier(), add_click_modifier()
Aliases for register_click_multiplier() and register_click_modifier().

unregister_click_modifier(modifier_id)
Removes a click modifier by key or by the local modifier ID for the current mod.

get_click_modifiers()
Returns metadata for active click modifiers. The default console command /clickmodifiers shows the same list.

8. Events and Timers
Mods can subscribe to game events instead of polling. Event callbacks receive one payload dict. If a callback raises an exception, the game prints an error and continues running. Event and timer callbacks run under the owning mod namespace, so objects they create are cleaned correctly during mod reloads.

on_event(event_name, callback)
Registers an event callback and returns a hook ID. Event names are normalized, so aliases like click, flesh_clicked, on_flesh_clicked, bought, save, and load work.

```
def register(game):
    def clicked(event):
        print("Click gains:", event["gains"])

    game.on_event("flesh_clicked", clicked)
```

off_event(hook_id)
Removes an event callback by hook ID.

Event helper aliases

Helper	Event
on_flesh_clicked(callback)	flesh_clicked
on_upgrade_bought(callback)	upgrade_bought
on_bought(callback)	upgrade_bought
on_upgrade_buy(callback)	upgrade_bought
on_save(callback)	save
on_load(callback)	load

Common events

Event	When it fires
flesh_clicked	After a click is processed and final currency gains are awarded.
upgrade_bought	After an upgrade is purchased, its count is increased, and on_buy effects are applied.
save	After the active save slot is written to disk.
load	After a save slot is loaded or applied during startup, recovery, or manual load.
save_created	After a new fresh save slot is created from the Saves tab or API.

Common event payloads

Event	Important payload fields
flesh_clicked	x, y, n_press, multiplier, vanilla_multiplier, crit_multiplier, base_gains, gains, modifiers, total_clicks
upgrade_bought	upgrade_id, upgrade, owned_before, owned_after, cost, cost_currency

Event	Important payload fields
save	save_id, save_name, slot, auto_backup
load	save_id, save_name, reason
save_created	save_id, save_name

set_timer(seconds, callback, repeat=True)
Runs a callback after a delay. If repeat is True, the callback keeps running until it returns False or is cancelled. Returns a timer ID.

set_timeout(seconds, callback)
Runs a callback once after a delay.

set_interval(seconds, callback)
Runs a callback repeatedly.

cancel_timer(timer_id), clear_timer(timer_id)
Stops a timer created by the mod timer API.

```
def register(game):
    def tick():
        print("bones:", game.get_currency("bones"))
        return True

    game.set_interval(1.0, tick)
```

9. Save Data and Mod Settings
Save namespaces are stored inside the active save slot. Use them for mod progress that should travel with a save. Mod settings are stored globally in settings.json. Use them for preferences that should apply across saves.

get_save_namespace(namespace=None)
Returns a dict stored under the current mod ID. If namespace is supplied, that namespace is used instead.

set_save_namespace(namespace=None, data=None)
Replaces a namespace dict and marks the active save dirty.

get_save_data(namespace=None), set_save_data(data, namespace=None)
Compatibility aliases for get_save_namespace() and set_save_namespace().

get_save_value(key, default=None, namespace=None)
Reads one value from a save namespace.

set_save_value(key, value, namespace=None)
Writes one value to a save namespace and marks the active save dirty.

delete_save_value(key, namespace=None)
Deletes one value from a save namespace if it exists.

```
def register(game):
    visits = game.get_save_value("visits", 0)
    game.set_save_value("visits", visits + 1)
```

get_mod_settings(namespace=None)
Returns a global settings dict for the current mod ID.

get_mod_setting(key, default=None, namespace=None)
Reads one global mod setting.

set_mod_setting(key, value, namespace=None)
Writes one global mod setting immediately.

delete_mod_setting(key, namespace=None)
Deletes one global mod setting if it exists.

```
def register(game):
    enabled = game.get_mod_setting("bonus_enabled", True)
    game.set_mod_setting("bonus_enabled", enabled)
```

10. Saves Tab and Required Mods
The game supports multiple save slots. The Saves tab can create a fresh save, manually save the active save, make backups, duplicate saves, rename saves, delete inactive saves, export saves, import saves, and restore backups.

When a save is written, the game records the loaded mods as required_mods. If a save or backup requires missing mods, the game warns before loading it:

```
This save file requires missing mods: [missing mods], loading may cause corruption or loss of data.
Load anyway?
```

Loading a modded save without its required mods can remove currencies, upgrades, achievements, or mod namespace data from the player experience. The data may still exist in the save JSON, but the game cannot show or use features from missing mods.

New saves created from the Saves tab are fresh saves. They do not clone the current currency amounts or owned upgrades.

11. Achievements

Achievements are milestones displayed in the Achievements tab. Only unlocked achievements are shown in the tab, and mod achievements are only shown when their source mod is loaded.

register_achievement(key, data)

Registers a new achievement, or updates an existing one if the key already exists. Source metadata is saved automatically.
- key - string. Unique internal ID. Example: "first_bone"
- data - dict with fields: name, desc, unlocked. Start unlocked as False.

```
game.register_achievement("first_bone", {
    "name": "Bonehead",
    "desc": "Obtain your first bone.",
    "unlocked": False,
})
```

unlock_achievement(key)

Unlocks an achievement by key. Safe to call even if already unlocked.

```
game.unlock_achievement("first_bone")
```

Built-in achievements are saved as builtin. Mod achievements are saved with source, mod_id, and mod_name so stale mod achievements are not displayed after the mod is removed or disabled.

12. Custom Tabs

Mods can add their own tabs to the notebook UI. Tabs can be populated with any GTK widgets. Built-in tab IDs are: "upgrades", "achievements", "leaderboard", "saves", "settings", "console", and "stats".

add_tab(tab_id, label, page_box=None)

Adds a custom tab to the notebook and returns the Gtk.Box page so you can populate it with widgets.
- tab_id - unique string ID. Used to reference this tab in add_tab_button() and add_image().
- label - text shown on the tab.
- page_box - optional Gtk.Box. If omitted, a vertical box with 6px spacing is created automatically.

Can be called before or after build_ui. If called before, the tab is queued and added when the UI is built. If called after, it is added immediately.

```
import gi
gi.require_version("Gtk", "4.0")
from gi.repository import Gtk

def register(game):
    box = game.add_tab("mystats", "My Stats")
    box.append(Gtk.Label(label="Hello from my mod!"))
```

add_tab_button(tab_id, label, callback)

Adds a button to any tab, built-in or mod-created. The callback receives the Gtk.Button as its only argument.

```
def register(game):
    def my_callback(button):
        print("clicked!")

    game.add_tab_button("settings", "Reset Stats", my_callback)
```

add_button(tab_id, label, callback) is a deprecated compatibility alias. It still works, but the mod is marked as legacy and receives a deprecation warning.

13. Main Image, Assets, and Sound

set_flesh_image(path)
Overrides the main clickable image with a custom file. If the GTK picture already exists, the new image is loaded immediately. Calling load_flesh_image() afterward is still safe but is no longer required for runtime changes.

```
def register(game):
    custom = os.path.join(MOD_DIR, "assets", "my_image.png")
    if os.path.exists(custom):
        game.set_flesh_image(custom)
```

load_flesh_image()
Reloads the clickable image from the current flesh_image_path.

set_click_sound(path)
Overrides the click sound. Must be a .wav file. Can be called at any time, and the new sound takes effect on the next click.

```
def register(game):
    game.set_click_sound(os.path.join(MOD_DIR, "assets", "pop.wav"))
```

resolve_asset_path(path, owner=None), resolve_mod_asset_path(path, owner=None)
Resolves an absolute path or a relative app/mod asset path. Relative mod paths check the mod folder, then its assets/ folder, then the application asset locations. resolve_mod_asset_path() is an alias.

```
icon_path = game.resolve_asset_path("assets/icon.png")
```

Default Asset Lookup
The default flesh.png and click.wav use the shared app asset resolver. On Windows, the resolver prefers _internal next to the game. On Linux, it prefers assets next to the game. Legacy same-folder and bundled paths are still checked as fallbacks.

Platform	Preferred default asset location
Windows	_internal/flesh.png and _internal/click.wav next to the game
Linux	assets/flesh.png and assets/click.wav next to the game

14. Mod Images and Sprites
Mods can add images inside normal tab layouts or create free-positioned sprites above the game window. Relative paths are resolved from the owning mod folder and assets/ directory.

add_image(tab_id, image_path, size=None, width=None, height=None, tooltip=None, expand=False, fit="contain")
Adds a Gtk.Picture to a built-in or mod-created tab and returns a ModImageHandle. size can be one number or a (width, height) pair. fit accepts contain, cover, fill, or scale-down.

```
def register(game):
    game.add_tab("gallery", "Gallery")
    logo = game.add_image(
        "gallery",
        "assets/logo.png",
        size=(128, 64),
        tooltip="My logo",
    )
```

create_image(image_path, size=(100, 100), pos=(0, 0), tab_id=None, ...)
Creates a free-positioned window sprite and returns a ModSpriteHandle. Passing tab_id sends the image through add_image() instead. width, height, x, and y can be used instead of size and pos.

create_sprite(...)
Alias for create_image().

```
def register(game):
    sprite = game.create_sprite(
        "assets/logo.png",
        size=(128, 64),
        pos=(20, 20),
    )
    sprite.set_velocity(3, 2)

    def animate():
```

```
    vx, vy = sprite.get_velocity()
    sprite.bounce(vx, vy)
    return True

game.set_interval(0.016, animate)
```

Sprites wait for valid GTK window bounds before moving.

ModImageHandle methods

Method	What it does
get_widget()	Returns the underlying GTK widget.
set_image(path)	Changes the image.
set_size(width=None, height=None, size=None)	Changes the requested size.
get_width(), get_height()	Returns the stored dimensions.
set_opacity(alpha), get_opacity()	Sets or reads opacity. Values above 1 are treated as percentages.
hide(), show(), set_visible(value), is_visible()	Controls visibility.
remove()	Removes the image and returns False if it was already removed.

ModSpriteHandle movement and rotation

Method	What it does
move_to(x, y), set_pos(x, y)	Moves to an absolute position.
move_by(dx, dy)	Moves by an offset.
get_pos(), get_x(), get_y()	Reads position.
set_x(x), set_y(y)	Changes one coordinate.
set_velocity(vx, vy), get_velocity()	Stores or reads velocity.
step(seconds=1.0)	Moves using the stored velocity.
set_rotation(angle), get_rotation(), rotate_by(delta)	Controls rotation in degrees.

ModSpriteHandle bounds and bouncing

Method	What it does
get_window_bounds()	Returns visible window width and height.
get_bounds()	Returns sprite position, dimensions, edges, and window dimensions.
get_edge_hit(margin=0)	Reports left, right, top, bottom, horizontal, and vertical edge contact.
get_corner_hit(margin=0)	Returns top-left, top-right, bottom-left, bottom-right, or None.
is_touching_corner(margin=0)	Returns True when touching a corner.
bounce(vx=None, vy=None, margin=0)	Moves, clamps to the visible window, reverses velocity at hit edges, and returns hit metadata.
bounce_velocity(...)	Alias for bounce().

get_sprite(sprite_id), hide_image(image), show_image(image), remove_image(image)

get_sprite() looks up a sprite by its generated ID. The image helpers accept a handle, and sprite IDs can also be used where supported. Mod-created images and sprites are removed automatically when mods reload.

PNG sprites use PyCairo surfaces. Non-PNG sprite formats use GdkPixbuf when available.

15. Game Title

Mods can offer a custom game title. The Settings mod list shows a Title switch next to mods that provide one. Only one title mod can be active at a time. Enabling one title switch disables the previously selected title switch.

game_title in manifest.json

```
{
  "name": "Console Tester",
  "version": "1.0.0",
  "description": "testing ingame console",
  "author": "xtexecute",
  "game_title": "Console Tester"
}
```

set_game_title(title)
Offers a custom window title from the currently loading mod. Call from register(game). The player still controls whether that title is used through the Settings switch.

```
def register(game):
    game.set_game_title("Bone Clicker")
```

16. Console Commands
The Console tab captures stdout and stderr and can run slash commands. Mods can register their own commands. Command callbacks run under the owning mod namespace.

register_console_command(name, callback, help_text="", aliases=None)
Registers a command. name and aliases do not need a leading slash. The callback receives args and raw when possible. If a callback only accepts args, the game falls back to that call shape.

```
def register(game):
    def heal(args, raw):
        amount = float(args[0]) if args else 1.0
        game.add_currency("flesh", amount)
        return f"Added {amount:g} flesh"

    game.register_console_command(
        "heal",
        heal,
        "Add flesh: /heal <amount>",
        aliases=["hf"],
    )
```

unregister_console_command(name)
Removes a command by command name or alias.

Default commands

Command	Aliases
/addcurrency <currency> <amount>	/ac, /add
/takecurrency <currency> <amount>	/tc, /take
/currencymodifier <currency> <multiplier>	/cm, /multcurrency, /multiplycurrency
/setcurrency <currency> <amount>	/sc
/currencies	/currencylist, /cl
/clickmodifiers	/modifiers, /clickmods
/apis	/api, /listapis
/saves	/listsaves, /lsaves
/save	/savegame
/reload	/reloadmods, /rl
/print <text>	/p, /echo
/help [command]	/h, /?, /commands

17. Mod Reloading and Cleanup
Fleshfetch can reload installed mods without restarting the application. Use /reload, or enable Auto reload mod changes in Settings to apply mod enable, disable, and security-dismissal changes immediately.

reload_mods()
Flushes the active save, removes mod-owned runtime objects, restores built-in registries and assets, loads mods again, refreshes the UI, and returns a summary string with loaded, paused, and disabled counts.

```
result = game.reload_mods()
print(result)
```

Objects cleaned during reload
- Mod timers and event hooks
- Click modifiers
- Shared APIs and their aliases
- Console commands and aliases

- Mod-created buttons, tab images, sprites, and custom tabs
- Mod currency, upgrade, title, image, sound, and ownership registries

Callbacks are executed under their owner mod namespace. This keeps objects created later by timers, events, click modifiers, console commands, and API cleanup methods attached to the correct mod for reload cleanup.

18. Deprecation Warnings

Deprecated APIs and legacy data formats still work where documented, but the mod is labeled (legacy) in Settings. Warning details are shown in yellow and printed to the console with [mods][deprecated].

Legacy usage	Preferred replacement
add_button(tab_id, label, callback)	add_tab_button(tab_id, label, callback)
Upgrade fpc and fps fields	currency_effects entries using cpc and cps
Upgrade type without category	category

Deprecation warnings do not pause the mod. They are separate from security warnings and missing dependency blocking.

19. Disabling Vanilla Content

These functions let a mod strip out built-in content. Call them at the start of register(), before adding replacement content.

disable_vanilla_achievements()

Removes all built-in achievements. Achievements registered by your mod after this call are unaffected.

```
def register(game):
    game.disable_vanilla_achievements()
    game.register_achievement("my_achievement", { ... })
```

disable_vanilla_upgrades()

Removes all built-in upgrades. Upgrades registered by your mod after this call are unaffected.

```
def register(game):
    game.disable_vanilla_upgrades()
    game.register_upgrade("my_upgrade", { ... })
```

disable_vanilla(primary_currency)

The nuclear option. Removes all built-in upgrades and achievements, hides the flesh currency from the UI, and sets a new primary currency. The saved vanilla flesh amount is kept intact.

- The new primary currency receives the base per-click gain instead of flesh.
- The new primary currency is shown in the stats panel.
- The saved vanilla flesh amount and legacy flesh counter are not overwritten by modded currency values.

```
def register(game):
    game.register_currency("souls", "Souls")
    game.disable_vanilla("souls")
    game.register_upgrade("soul_shard", {
        "name": "Soul Shard",
        "desc": "+1 soul/sec.",
        "category": "auto",
        "base_cost": 25,
        "cost_mult": 1.15,
        "cost_currency": "souls",
        "currency_effects": [
            {"currency": "souls", "cpc": 0.0, "cps": 1.0, "on_buy": 0.0},
        ],
    })
```

disable_vanilla() calls disable_vanilla_upgrades() and disable_vanilla_achievements() internally. You do not need to call all three separately.

20. Full Example Mod

A complete mod folder demonstrating currencies, upgrades, achievements, custom tabs, images, a moving sprite, sound, click modifiers, save data, settings, events, a shared API, console commands, and enabled.txt.

Folder structure

```
mods/
  my_mod/
```

```
mod.py
manifest.json
enabled.txt
assets/
  custom.png
  pop.wav
```

manifest.json

```
{
  "name": "My Mod",
  "version": "2.1.0",
  "description": "Adds bones, a custom tab, images, sound, and click effects.",
  "author": "you",
  "game_title": "Bone Clicker"
}
```

mod.py

```
import os
import gi

gi.require_version("Gtk", "4.0")
from gi.repository import Gtk

class BonesAPI:
    def __init__(self, game):
        self.game = game

    def amount(self):
        return self.game.get_currency("bones")

def register(game):
    game.register_currency("bones", "Bones")
    game.set_game_title("Bone Clicker")

    game.register_api(
        "bones",
        BonesAPI(game),
        version="1.0.0",
        description="Read the current bone amount.",
    )

    game.register_upgrade("bone_grinder", {
        "name": "Bone Grinder",
        "desc": "+1 bone/sec.",
        "category": "auto",
        "base_cost": 50,
        "cost_mult": 1.2,
        "cost_currency": "flesh",
        "currency_effects": [
            {"currency": "bones", "cpc": 0.0, "cps": 1.0, "on_buy": 0.0},
        ],
    })

    game.register_achievement("first_bone", {
        "name": "Bonehead",
        "desc": "Obtain your first bone.",
        "unlocked": False,
    })

    img = os.path.join(MOD_DIR, "assets", "custom.png")
    if os.path.exists(img):
        game.set_flesh_image(img)

    snd = os.path.join(MOD_DIR, "assets", "pop.wav")
    if os.path.exists(snd):
        game.set_click_sound(snd)
```

```
tab_box = game.add_tab("bones_tab", "Bones")
status_label = Gtk.Label(label="Bones: 0")
tab_box.append(status_label)
game.add_image("bones_tab", "assets/custom.png", size=(128, 64))

sprite = game.create_sprite("assets/custom.png", size=(64, 32), pos=(20, 20))
sprite.set_velocity(2.5, 1.5)

game.register_click_multiplier(
    "lucky_bones",
    chance=10,
    multiplier=2.0,
    currency="bones",
    description="10 percent chance to double bone click gains.",
)

def reset_bones(button):
    game.set_currency("bones", 0.0)
    print("Bones reset!")

game.add_tab_button("settings", "Reset Bones", reset_bones)

def tick():
    bones = game.get_currency("bones")
    status_label.set_text(f"Bones: {int(bones)}")
    game.set_save_value(
        "highest_bones",
        max(game.get_save_value("highest_bones", 0), bones),
    )
    if bones >= 1:
        game.unlock_achievement("first_bone")
    vx, vy = sprite.get_velocity()
    sprite.bounce(vx, vy)
    return True

game.set_interval(0.05, tick)

def report(args, raw):
    return f"Bones: {game.get_currency('bones'):g}"

game.register_console_command("bones", report, "Show bones.")
```

21. Quick Reference

Common mod API functions

Function	What it does
register_currency(name, display)	Add a new currency to the game.
get_currency(name)	Read the player current amount.
set_currency(name, value)	Set a currency to an exact value.
add_currency(name, amount)	Add or subtract a currency amount.
register_upgrade(uid, data)	Add or update an upgrade.
register_achievement(key, data)	Add or update an achievement.
unlock_achievement(key)	Unlock an achievement.
register_click_modifier(...)	Add a custom click rule.
register_click_multiplier(...)	Add a chance-based click multiplier.
on_event(name, callback)	Subscribe to a game event.
off_event(hook_id)	Remove an event subscription.
set_timer(...), set_timeout(...), set_interval(...)	Create mod-owned timers.
cancel_timer(id), clear_timer(id)	Stop a mod timer.
get_save_value(...), set_save_value(...)	Read and write save data.
get_mod_setting(...), set_mod_setting(...)	Read and write global mod settings.
has_mod(...), require_mod(...)	Check or require another mod.

Function	What it does
register_api(...), require_api(...)	Provide or consume a shared API.
list_apis(), get_api_info(...)	Inspect shared APIs.
add_tab(...), add_tab_button(...)	Create a custom tab or button.
add_image(...), create_sprite(...)	Create tab images or moving sprites.
hide_image(...), show_image(...), remove_image(...)	Control mod-created images.
set_flesh_image(path)	Replace the clickable image.
set_click_sound(path)	Replace the click sound.
set_game_title(title)	Offer a selectable title.
register_console_command(...)	Add a console command.
reload_mods()	Reload installed mods and clean runtime objects.
disable_vanilla(primary_currency)	Replace vanilla content and primary currency.

Image handle methods

Method	What it does
set_image(path)	Change the image.
set_size(...), get_width(), get_height()	Change or read dimensions.
set_opacity(...), get_opacity()	Change or read opacity.
hide(), show(), set_visible(...), is_visible()	Control visibility.
remove()	Remove the image.

Sprite-only methods

Method	What it does
move_to(...), move_by(...), set_pos(...)	Move the sprite.
get_pos(), get_x(), get_y(), set_x(), set_y()	Read or change coordinates.
set_velocity(...), get_velocity(), step(...)	Store velocity and move from it.
set_rotation(...), get_rotation(), rotate_by(...)	Control rotation.
get_bounds(), get_window_bounds()	Read sprite and window bounds.
get_edge_hit(), get_corner_hit(), is_touching_corner()	Detect edge and corner contact.
bounce(...), bounce_velocity(...)	Move and bounce inside the visible window.

Built-in Tab IDs

Use these IDs with add_tab_button() and add_image() to target built-in tabs:

Tab ID	Description
upgrades	The upgrades shop tab.
achievements	The achievements tab.
leaderboard	The online leaderboard tab.
saves	The save management tab.
settings	The settings and installed mods tab.
console	The in-game console tab.
stats	The stats tab.

22. Custom Save Types

Fleshfetch v1.3.2 adds save types so mods can keep specialized saves separate from normal game saves. The default save type is singleplayer. The built-in Saves tab only displays and imports that type.

Creating a custom save

Pass a save type when creating a fresh slot. Save type names are normalized to lowercase and may contain letters, numbers, hyphens, and underscores.

```
slot = game.create_save_slot("Shared World", save_kind="my_mod_world")
```

Importing a custom save

Use expected_save_kind when importing through mod code. The import fails cleanly if the selected file belongs to a different save type.

```
ok, message, slot = game.import_save_slot(
    path,
    expected_save_kind="my_mod_world",
)
```

Save behavior

- A custom save keeps its type when duplicated, exported, imported, backed up, or restored.
- Custom save types do not overwrite the legacy singleplayer state or achievement files.
- At least one normal singleplayer save must remain, but custom save types can manage their own slot rules.
- Existing saves without a save type are treated as singleplayer saves for compatibility.

Event payloads

The save, load, and save_created event payloads now include save_kind. Mods can use it to ignore saves that belong to another system.

Event	New field
save	save_kind
load	save_kind
save_created	save_kind

```
def register(game):
    def loaded(event):
        if event["save_kind"] != "my_mod_world":
            return
        print("Custom save loaded")
    game.on_load(loaded)
```